

La siguiente tabla muestra los componentes del sistema, la cantidad de componentes usados y el MTTF en horas:

Componente	Cantidad	MTTF
CPU	1	1.000.000
Placa Base	1	200.000
DIMM de memoria	4	500.000
Fuente alimentación	1	100.000

e) **Calcula** el tiempo medio hasta fallo del sistema (MTTF)

$$\text{MTTF} = 1/(1/1e6 + 1/2e5 + 4/5e5 + 1/1e5) = 41667 \text{ Horas.}$$

Esta CPU tiene direcciones lógicas de 48 bits y direcciones físicas de 32 bits. La jerarquía de memoria tiene las siguientes características.

- Cache L1 tiene un tamaño de 16 Kbytes y es 4-asociativa. La política de escritura es Write through-Write NO Allocate. El tamaño de línea es de 128 bytes.
- TLB de 16 entradas completamente asociativo.
- El tamaño de página del sistema es de 8 KBytes.

f) **Calcula** el número de líneas, vías y conjuntos que tiene la cache L1. **Especifica claramente** cómo has realizado los cálculos.

$$16 * 1024 \text{ Bytes} / 128 \text{ Bytes} = 128 \text{ líneas.}$$

$$4\text{-asociativa} \rightarrow 4 \text{ líneas por conjunto} \rightarrow 4 \text{ vías.}$$

$$128 \text{ líneas} / 4 \text{ vías} = 32 \text{ conjuntos}$$

La CPU lanza un acceso a la dirección lógica 0xCD0123456789, que el TLB traduce como 0x3BCDE789.

g) **Indica** en qué conjunto de la L1 se encuentra el dato y cuál es la etiqueta que se guardará en memoria cache L1. **Justifica** la respuesta.

Líneas de 128 bytes $\rightarrow$ 7 bits de byte

32 conjuntos $\rightarrow$ 5 bits

@física $\rightarrow$ | TAG (20 bits) | Cjto (5 bits) | byte (7 bits) |

Cjto = 15 (F en hexa)

Etiqueta = 0x3BCDE

h) **Indica** el tamaño máximo que puede tener la cache L1 para que sea posible acceder a la cache L1 y al TLB en paralelo, suponiendo que se mantienen el resto de parámetros de la jerarquía de memoria. **Justifica** la respuesta.

El tamaño máximo de la cache viene delimitado por los bits de línea + los bits de conjunto. Estos bits pueden direccionar como máximo una página del sistema (8 KBytes).

8k (pagina) $\rightarrow$ bits de línea + bits de conjunto = 13 $\rightarrow$ 1 vía = 8K

Tamaño cache = Tamaño vía * asociatividad = $8K * 4 = 32K$ bytes

- c) **Traduce** a ensamblador del x86 el siguiente código, suponiendo que está dentro de la función **examen**. Utiliza registros que no hayas de salvar/restaurar. Responde usando un máximo de 10 instrucciones.

```
i=0;
while(i<y && aux.c2[i] != '.')
{
    aux.c2[i] = '#';
    i++;
}
```

```
MOVW $0, -4(%ebp)          # i=0;
MOVW 16(%ebp), %dx          # y en %dx
while: CMPW %dx, -4(%ebp)    # i < y
    JGE fin_while
    MOVSWL -4(%ebp), %eax     # extension_signo (i) en %eax
    CMPB $.', -74(%ebp, %eax) # aux.c2[i] != '.' --> c2: %ebp -76 + 2(a)
    JE fin_while
    MOVW $. '#', -74(%ebp, %eax) # aux.c2[i] = '#'
    INCW -4(%ebp)            # i++;
    JMP while
fin_while:
```

- d) **Traduce** la siguiente sentencia a ensamblador del x86, suponiendo que está dentro de la función **examen**. Responde usando un máximo de 8 instrucciones.

```
v1[5] = F1(v1, *x, x);
```

```
PUSHL 12(%ebp)             # int *x es el 2o. parámetro de examen
MOVL 12(%ebp), %eax
PUSHL (%eax)                # *x es el contenido de ese 2o. parámetro
LEAL -44(%ebp), %eax
PUSHL %eax                  # @_inicio de v1
CALL F1
ADDL $12, %esp              # eliminar parámetros
MOVL %eax, -24(%ebp)        # recoger/usar el resultado: v1[5] en %ebp -44 + 4B*5
```